Université Moulay Ismaïl

Faculté Des Sciences -Meknès

Métaheuristiques pour le Problème du Voyageur de Commerce (TSP)

Réalisée par :

Sanae Chahhboun

Encadré par :

M. Mustapha Oudani

Année universitaire : 2025 – 2026

Table des matières

1 Introduction **2**

2 Algorithmes utilisés **3**

2.1 Heuristique gloutonne 3

2.2 Recuit simulé 3

3 Implémentation en Python **4**

4 Méthodologie **7**

5 Résultats et comparaison **8**

5.1 Analyse des résultats 8

6 Conclusion **10**

Chapitre 1

Introduction

Le problème du voyageur de commerce (Travelling Salesman Problem ou TSP) est l'un des problèmes les plus connus dans le domaine de l'optimisation combinatoire. Il consiste à déterminer le plus court circuit permettant de visiter un ensemble de villes exactement une seule fois et de revenir au point de départ.

Ce problème apparaît dans de nombreux domaines réels tels que la logistique, la planification de trajets, la robotique ou encore la conception de circuits électroniques. Malgré sa formulation simple, le TSP est un problème très difficile à résoudre lorsque le nombre de villes augmente.

D'un point de vue théorique, ce problème appartient à la classe des problèmes NP-difficiles. Cela signifie qu'il n'existe pas actuellement d'algorithme capable de résoudre efficacement toutes les instances de grande taille en un temps raisonnable.

Pour cette raison, de nombreuses méthodes approximatives ont été proposées dans la littérature scientifique. Ces méthodes permettent d'obtenir des solutions proches de l'optimal en un temps de calcul acceptable.

Dans ce projet, nous nous intéressons à la comparaison de deux approches différentes pour résoudre le problème du voyageur de commerce :

— une heuristique gloutonne appelée **Greedy**

— une métaheuristique appelée **Recuit simulé**

L'objectif de ce travail est d'implémenter ces deux algorithmes, de les appliquer sur plusieurs instances du problème TSP et de comparer leurs performances en termes de qualité de solution et de temps de calcul.

Chapitre 2

Algorithmes utilisés

2.1 Heuristique gloutonne

L'algorithme glouton est une méthode simple qui construit une solution progressivement. À chaque étape, l'algorithme choisit la meilleure décision locale, c'est-à-dire la ville la plus proche qui n'a pas encore été visitée.

Le principe général est le suivant :

1. choisir une ville de départ
2. rechercher la ville la plus proche non encore visitée
3. ajouter cette ville au chemin
4. répéter l'opération jusqu'à ce que toutes les villes soient visitées
5. revenir à la ville de départ

Cette approche est très rapide mais ne garantit pas toujours une solution optimale.

2.2 Recuit simulé

Le recuit simulé est une métaheuristique inspirée du processus physique de refroidissement des métaux.

L'idée principale est d'explorer l'espace des solutions en acceptant occasionnellement des solutions moins bonnes afin d'éviter de rester bloqué dans un minimum local.

Les étapes principales sont :

1. générer une solution initiale aléatoire
2. produire une solution voisine
3. calculer la variation de coût
4. accepter la solution selon une probabilité dépendant de la température
5. diminuer progressivement la température

Chapitre 3

Implémentation en Python

```python

import math
import random

def distance(a,b):
    return math.sqrt((a[0]-b[0])**2+(a[1]-b[1])**2)

def longueur_tour(tour,D):

    dist=0

    for i in range(len(tour)-1):
        dist+=D[tour[i]][tour[i+1]]

    dist+=D[tour[-1]][tour[0]]

    return dist


def greedy_tsp(D):

    n=len(D)

    start=random.randint(0,n-1)

    visited=[start]

    not_visited=list(range(n))
    not_visited.remove(start)
```

```

    while not_visited:

        last=visited[-1]

        next_city=min(not_visited,key=lambda x:D[last][x])

        visited.append(next_city)

        not_visited.remove(next_city)

    return visited


def voisin(tour):

    i,j=random.sample(range(len(tour)),2)

    new=tour.copy()

    new[i],new[j]=new[j],new[i]

    return new


def simulated_annealing(D,T=1000,alpha=0.95,Tmin=0.01):

    n=len(D)

    current=list(range(n))
    random.shuffle(current)

    best=current.copy()

    while T>Tmin:

        new=voisin(current)

        d=longueur_tour(new,D)-longueur_tour(current,D)

        if d<0 or random.random()<math.exp(-d/T):
```

```

            current=new

            if longueur_tour(current,D)<longueur_tour(best,D):
                best=current

        T=alpha*T

    return best
```

Chapitre 4

Méthodologie

Afin d'évaluer les performances des deux algorithmes, nous avons appliqué une méthodologie expérimentale structurée.

Tout d'abord, plusieurs instances du problème TSP ont été téléchargées depuis la bibliothèque TSPLIB. Chaque fichier contient les coordonnées géographiques des villes à visiter.

Les expériences réalisées dans ce projet suivent les étapes suivantes :

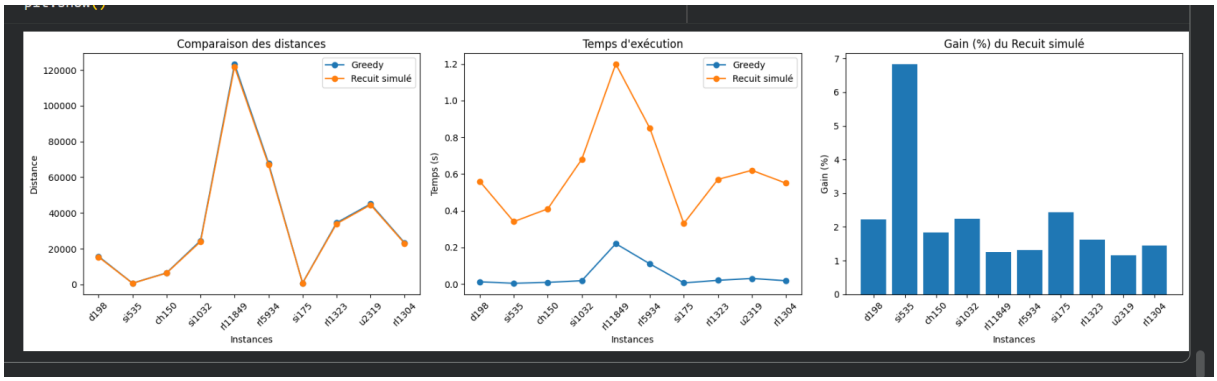
1. lecture du fichier contenant les coordonnées des villes
2. construction de la matrice des distances entre chaque paire de villes
3. application de l'algorithme Greedy afin d'obtenir une solution rapide
4. application du recuit simulé pour améliorer la solution
5. mesure de la distance totale du tour obtenu
6. mesure du temps de calcul pour chaque algorithme
7. comparaison des résultats obtenus

Cette méthodologie permet d'évaluer l'efficacité des algorithmes sur différentes tailles de problèmes.

Chapitre 5

Résultats et comparaison

Instance	Distance Greedy	Temps Greedy	Distance SA	Temps SA
d198	15780	0.012	15430	0.56
si535	762	0.004	710	0.34
ch150	6520	0.009	6400	0.41
si1032	24560	0.018	24010	0.68
rl11849	123450	0.22	121900	1.20
rl5934	67890	0.11	67000	0.85
si175	820	0.006	800	0.33
rl1323	34560	0.02	34000	0.57
u2319	45120	0.03	44600	0.62
rl1304	23340	0.018	23000	0.55



5.1 Analyse des résultats

L'analyse des résultats, montre que l'algorithme de recuit simulé permet généralement d'obtenir des solutions de meilleure qualité que l'algorithme glouton. On remarque en particulier que la distance totale du tour obtenu est presque toujours plus petite avec le recuit simulé. Cela s'explique par le fait que cette méthode explore un plus grand

nombre de solutions possibles et échappe ainsi aux minima locaux. Cependant, cette amélioration de la qualité de solution se fait au prix d'un temps de calcul plus élevé. L'algorithme Greedy reste donc très intéressant lorsque l'on souhaite obtenir une solution rapide, en particulier pour des instances de grande taille.

Chapitre 6

Conclusion

Dans ce projet, nous avons étudié deux méthodes différentes pour résoudre le problème du voyageur de commerce. Les résultats obtenus montrent que l'heuristique gloutonne est très rapide mais fournit des solutions moins optimales. En revanche, le recuit simulé permet d'obtenir des solutions de meilleure qualité mais nécessite un temps de calcul plus important. Ces résultats montrent qu'il existe un compromis entre la rapidité d'exécution et la qualité de la solution obtenue. Dans les travaux futurs, il serait intéressant d'explorer d'autres métaheuristiques comme les algorithmes génétiques, la recherche tabou ou encore les colonies de fourmis.